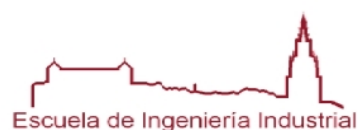


Práctica 4: Comunicación entre PC utilizando Python y nodeMCU.

Escuela de Ingeniería Industrial de Toledo

Autor

Juan Moreno García



Índice general

4. Programación de Arduino	2
4.1. Objetivos.	2
4.2. Una primera aplicación: un chat unidireccional.	2
4.3. nodeMCU cliente - Python servidor.	4
4.4. Python cliente - nodeMCU servidor.	6

4

Programación de Arduino

4.1. Objetivos.

Esta práctica pretende explicar al alumno la programación de la comunicación entre un programa *Python* y un *Arduino* o *nodeMCE* utilizando el cable que conecta el mismo al PC. Dado que *Python* es multiplataforma y el lenguaje de programación de *Arduino* es compatible con multitud de dispositivos, y es una forma muy general de realizar la comunicación entre diferentes plataformas vía RS-232. Al final de la práctica el alumno debe ser capaz de:

1. Crear programas para comunicar un programa *Python* con un programa cargado en un *nodeMCU* o *Arduino*.
2. Crear aplicaciones con comunicación PC-*nodeMCU* vía puerto RS-232 utilizando el modelo cliente-servidor.

4.2. Una primera aplicación: un chat unidireccional.

En esta práctica se presentará la comunicación entre dos programas, un programa *Python* y un programa en *C* en el *nodeMCU*. El programa *Python* envía un mensaje leído desde el teclado al *nodeMCU*, que lo recibe y se lo devuelve al programa *Python* añadiéndole '[' y ']' como primer y último carácter de la secuencia devuelta.

El programa *Python* envía y recibe utilizando las funciones *sendMensaje()* y *receiveMensaje()* detalladas en la práctica 2. Por esto se debe incluir el archivo *mensaje.py* utilizado en dicha práctica en la misma carpeta donde se encuentre el archivo con el código anterior. El código de este programa es el siguiente:

```
1 import serial # Importamos la libreria de PySerial
2 from mensaje import *
```

```

3 PuertoSerie = abrirPort( 'COM11' ) #Abrimos el puerto a 115200
4 # Creamos un bucle sin fin
5 while True:
6     s = input('Envía un comentario: ')
7     sendMensaje(PuertoSerie,s) # Envía un mensaje (mensaje.py)
8     print( 'Esperando recibir ... ' )
9     r = receiveMensaje(PuertoSerie) # Recibe un mensaje (mensaje.py)
10    print( 'Respuesta recibida: ', r )

```

El módulo `mensaje` contiene las funciones `abrirPort()`, `sendMensaje()` y `receiveMensaje()`, para cerrar el puerto se debe utilizar la llamada `close()` del módulo `serial`. Se observa que este módulo hace uso de las funciones `write()` y `readline()` de la librería `serial` para implementar las funciones `sendMensaje()` y `receiveMensaje()`. El código de este módulo es el siguiente:

Archivo `mensaje.py`

```

1 import serial
2
3 def abrirPort(p):
4     return serial.Serial(
5         port=p, baudrate=115200, parity=serial.PARITY_NONE,
6         stopbits=serial.STOPBITS_ONE, bytesize=serial.EIGHTBITS
7     )
8
9 def sendMensaje(ps,s):
10    ps.write(s.encode('utf-8'))
11
12 def receiveMensaje(ps):
13    r = 'chg_'
14    while r[0:4]=='chg_': # Elimina mensajes de sincronización del nodeMCU
15        r = ps.readline().decode()[0:-2]
16    return r

```

El programa `C` a cargar en el `nodeMCU` comprueba si se ha recibido algo evaluando la condición `Serial.available()>0`, y en caso afirmativo recibe la trama hasta recibir un `'\n'`, y la reen vía entre corchetes utilizando la función `Serial.println`.

```

1 void setup() {
2     Serial.begin(115200);
3 }
4
5 void loop() {
6     String s;
7     if(Serial.available() > 0) {
8         s = Serial.readStringUntil('\n'); // recibe info hasta '\n'
9         // Envía lo recibido entre corchetes
10        Serial.println( String "[" + s + String "]" );
11    }
12    delay(10); // Para parar un poco el nodeMCU, no necesario
13 }

```

A modo de resumen destacar que el programa *Python* hace uso de las primitivas *write()* y *readline()* de la librería *serial* de *Python*, y el programa del *nodeMCU* utiliza las funciones *Serial.readStringUntil()* y *Serial.println()*. Todas eran ya conocidas excepto la función *Serial.readStringUntil()* que devuelve una cadena leída hasta un carácter de fin indicado como parámetro.

4.3. nodeMCU cliente - Python servidor.

En esta sección se presenta un ejemplo en el que el *nodeMCU* hace de cliente recibiendo las órdenes mediante un mando infrarrojo, recibe un comando y lo envía al programa *Python*, éste lo recibe, le añade al final '_RR' (ese será su "servicio") y se lo devuelve al *nodeMCU* que le vuelve a devolver el mensaje recibido a modo de reconocimiento.

El programa *Python* es más sencillo. Hace uso de las funciones *sendMensaje()* y *receiveMensaje()* del módulo *mensaje.py* ya explicadas anteriormente. Primero recibe un mensaje (línea 9), después envía la cadena recibida con '_RR' (línea 13), destacar que se le añade '\n' como carácter final. Para terminar recibiendo la cadena enviada (línea 16).

```

1  # Importamos la libreria de PySerial y mensaje
2  import serial
3  from mensaje import *
4
5  # Abrimos el puerto del arduino a 115200
6  PuertoSerie = abrirPort( 'COM3' )
7  while True: # Creamos un bucle sin fin
8      print("Esperando...")
9      r = receiveMensaje(PuertoSerie)
10     print( 'Respuesta recibida: ', r )
11
12     # añade "_RR" y envía
13     sendMensaje(PuertoSerie,r+'_RR\n')
14
15     # Espera mensaje de confirmación
16     r1 = receiveMensaje(PuertoSerie)
17     print( 'Respuesta recibida segunda: ', r1 )

```

El programa *nodeMCU* que comunica con el anterior es el siguiente:

```

1  #define LED 5 // pin D1 del nodeMCU
2  #include <IRremoteESP8266.h>
3  #include <IRrecv.h>
4
5  unsigned long claves[21] = {0xFFA25D,0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD,
6  0xFFC23D, 0xFFE01F,0xFFA857,0xFF906F,0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF,
7  0xFF18E7, 0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5,0xFF52AD};
8
9  String valor[21] = {"ON/OFF", "MODE", "Vo1OFF", "PLAY/PAUSE", "PREV", "NEXT",
10 "EQ", "-", "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

```

```
11
12 int RECV_PIN = 4; // pin 1 del receptor IR al pin 2 de Arduino
13
14 IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
15 decode_results results; // Crea objeto de 'decode_results'
16
17 void setup() {
18     pinMode(LED, OUTPUT); // LED como salida
19     Serial.begin(115200);
20     irrecv.enableIRIn(); // Inicializa el receptor
21 }
22
23 void loop() {
24     String s, r = "";
25     if (irrecv.decode(&results)) { // Si alguna señal recibida
26         // enciende el LED
27         digitalWrite(LED, HIGH);
28
29         // Lee el código pulsado y envía a programa python
30         s = leerHastaRetorno();
31         Serial.println( s );
32
33         // Recibir respuesta
34         r = leerLinea(); // recibe info hasta '\n'
35         // Envía lo recibido terminado en "\n"
36         Serial.println( r ); // devolver respuesta
37
38         // apaga el LED
39         digitalWrite(LED, LOW);
40     }
41 }
42 // Lee pulsaciones del mando hasta que se pulsa "U/SD"
43 String leerHastaRetorno() {
44     bool b = false;
45     String r="", s;
46     while (!b) {
47         if (irrecv.decode(&results)) { // Si alguna señal
48             // Lee el código pulsado y envía a programa python
49             s = getValor(results.value);
50             if (s!="U/SD") {
51                 // Enviar identificador pulsado a python
52                 if (s=="LOOP") r = r + String(":");
53                 else r = r + s;
54             } else {
55                 b = true; // completada cadena
56             }
57             irrecv.resume(); // Recibir el siguiente valor
58         }
59         // Necesario para evitar el problema del "Soft WDT reset"
60         // caused by watchdog timer.
61         delay(1); //
```



```

62         }
63         return String(r);
64     }
65
66     String leerLinea() {
67         // Espera hasta que haya algo que leer
68         // BUCLE TOTALMENTE NECESARIO
69         while(Serial.available() < 1) {
70             delay(1);
71         }
72         return Serial.readStringUntil('\n'); // Recibe info hasta '\n'
73     }
74
75     String getValor(unsigned long c) {
76         byte i;
77         i=0;
78         while( (i<21) && (claves[i]!=c) ) {
79             i++;
80         }
81         if (i==21) return String("");
82         else return String(valor[i]);
83     }

```

El programa tiene las funciones *leerHastaRetorno()* que lee códigos del mando hasta que recibe 'U/SD', además, cuando recibe 'LOOP' añade ':' a la secuencia, el resto de pulsaciones aparece tal cual las devuelve la función *getValor()*, ya conocida. Otra función nueva que aparece es *leerLinea()* que lee un mensaje hasta que llega el carácter '\n'. De esta función es destacable que hay un bucle, totalmente necesario, que bloquea el programa hasta que se recibe algún valor. La función *Serial.readStringUntil()* no es bloqueante, por lo que si no se pone el bucle entre las líneas 69 y 71 el programa continua y no funciona correctamente.

El *loop()* espera hasta que se lee una secuencia de pulsaciones del mando infrarrojo terminada en "U/SD" (línea 30), envía la secuencia como mensaje al programa *Python* (línea 31 que sincroniza con la línea 9 del programa *Python*), se queda esperando a recibir un mensaje de éste (línea 34, que sincroniza con la línea 13 del programa *Python*), y finalmente, devuelve el mensaje recibido al programa *Python* (línea 36 que sincroniza con la línea 16 del programa *Python*).

4.4. Python cliente - nodeMCU servidor.

En esta sección se presentarán dos programas, uno de ellos cargado en el *nodeMCU*, que presentan una arquitectura cliente-servidor. El programa *Python* será el cliente y el servidor estará cargado en el *nodeMCU*. El servidor atenderá dos posibles peticiones del cliente:

1. Realizar la suma de N enteros.
2. Incrementará tres *float* enviados desde el cliente una cantidad M , que es también un *float*.

Para poder atender estos dos servicios se ha utilizado el siguiente formato de trama:

$$opc : N : v_1 : \dots v_N : M \quad (4.1)$$

donde opc representa la petición (1 o 2), v_i es el valor en la posición i del vector entero o float (según la petición), y M es el *float* utilizado en el segundo servicio, y no es necesario en el primer servicio.

El programa *Python* consta de una función llamada *menu()* que muestra el menú al usuario para seleccionar el servicio y devuelve la opción elegida como *string*. Además, tiene el siguiente código principal:

```

1  # Importamos la libreria de PySerial
2  import serial
3  from mensaje import *
4
5  'Muestra el menú de las opciones y devuelve la seleccionada'
6  def menu() :
7      print("0) Salir")
8      print("1) Sumar N números enteros")
9      print("2) Incrementar 3 números float en M (float) cada uno")
10     return input('Elige una opción: ')
11
12  # Abrimos el puerto del arduino a 115200
13  PuertoSerie = abrirPort( 'COM4' ) # Cambiar si necesario
14
15  opc = '-1' # inicializado a un valor no válido
16  # Creamos un bucle sin fin
17  while opc!='0':
18      opc = menu() # muestra menú y devuelve opción en opc
19      r = opc + ':' # El primer valor de la trama es la opción elegida
20      if opc=='1': # Si es el caso 1
21          N = input('Dame el valor de N: ') # valor N como cadena
22          r += N # Añade N en la trama
23          for e in range(int(N)): # N valores a la trama
24              r += ':' + input('Dame un entero: ')
25      elif opc=='2': # Si es el caso 2
26          r += '3' # Añade 3 en la trama
27          for e in range(3): # añade los 3 valores a la trama
28              r += ':' + input('Dame un float: ')
29          r += ':' + input('Dame el valor de M (float): ') # M a la trama
30          r += '\n' # añade carácter final a la trama
31          print(r[:-1]) # formato trama opc:N:v1...vN:M, M sólo en caso 2
32      if opc>='1' and opc<='2': # Si es una opción válida
33          # envía trama
34          sendMensaje(PuertoSerie, r) # Envía la trama al nodeMCU
35          s = receiveMensaje(PuertoSerie) # Recibe la trama de respuesta
36          print( s ) # Muestra por pantalla la respuesta

```

El programa abre el puerto (línea 13), y mediante un bucle del que se sale tecleando la opción '0' (línea 17), presenta el menú y lee la entrada (línea 18). Posteriormente va

completando la trama en una cadena con el formato de la Ecuación 4.1 entre la líneas 18 y 36. Las líneas 18 y 19 son comunes a ambas peticiones y, según la opción elegida, se ejecutará el código entre las líneas 21 y 24 para la petición 1, o las líneas entre la 26 y 31, para la petición 2.

El programa servidor es el que se carga en el *nodeMCU* y contiene las siguientes funciones:

1. **cortaTrozo()** (línea 53): Función que dada la trama (parámetro *s*) y el inicio desde donde buscar (*inic*) devuelve un *String* con el trozo de la trama comprendido entre el carácter indicado con el índice *inic* y la primera aparición del carácter *car* (parámetro). En el parámetro *fin* (por referencia, importante) coloca el valor donde continuar buscando los siguientes campos de la trama. El parámetro *corta_si_no_esta* se utiliza para seleccionar la devolución cuando *car* no se encuentra en la parte a buscar. Si es *true* se devuelve el trozo desde *inic* hasta el final, si es *false* se devuelve “-1” como código de error.
2. **obtenListaEnteros()** (línea 63): Función que dada la trama recibida (parámetro *línea*) y el inicio desde donde buscar (*inic*) devuelve un vector de enteros de tamaño *N* y en el parámetro *fin* (por referencia, importante) coloca el valor donde continuar buscando los siguientes campos de la trama. Importante recordar que en algún punto del programa se debe liberar la memoria reservada en esta función.
3. **obtenListaFloat()** (línea 75): Función que dada la trama recibida (parámetro *línea*) y el inicio desde donde buscar (*inic*) devuelve un vector de float de tamaño *N* y en el parámetro *fin* (por referencia, importante) coloca el valor donde continuar buscando los siguientes campos de la trama. Similar a la anterior, pero devuelve un vector de enteros, no de *float*.
4. **sumaListaEnteros()** (línea 88): Función que dado un vector de *enteros* de tamaño *tam* devuelve la suma de los mismos. Realiza el cálculo pedido en la petición 1.
5. **incrementaListaEn()** (línea 96): Función que dado un vector de *float* de tamaño *tam* incrementa cada uno de sus valores en *M*. Realiza el cálculo pedido en la petición 2.
6. **leerTrama()** (línea 104): Es la función *leerLinea()* del ejemplo anterior parametrizado con el carácter fin de lectura.

El programa principal realiza las siguientes acciones:

- Comienza leyendo la trama (línea 14) y después corta la cadena que aparece antes del primer ‘.’ (es el código de la petición) utilizando la función *cortaTrozo()* (línea 16). Actualiza la variable *inic* que indica donde empieza el siguiente campo a obtener y almacena en *opc* el valor leído como entero utilizando la función *toInt()* del *String* de *Arduino*.
- A continuación lee el campo *N* (líneas 20-22), nuevamente se utiliza la función *cortaTrozo()* (línea 20).
- Líneas 24-29: En caso ser la petición 1 (*opc==1*) se deben leer *N* campos consecutivos y almacenarlos en un vector (también se puede hacer sin almacenarlo, pero es más interesante para el aprendizaje de esta forma). La función *obtenListaEnteros()* devuelve

un vector (memoria dinámica) con dichos elementos (línea 26), posteriormente se realiza la suma pedida mediante la función `sumaListaEnteros()` (línea 27) y se envía (línea 28). Finalmente, se libera la memoria dinámica devuelta por la función `obtenListaEnteros()`, este paso no se puede olvidar.

- Líneas 30-45: En la petición 2 se deben leer N float consecutivos (función `obtenListaEnteros()` en línea 32), y obtener el valor M (líneas 35-37). Esta primera parte es muy similar al caso anterior. Finalmente, se debe hacer el cálculo de la petición mediante la función `incrementaListaEn()` (línea 38), hacer el montaje de la trama a devolver (líneas 39 a 43), y que tiene el formato: $v_1 : \dots : v_n$.

```

1  #define LED 5 // pin D1 del nodeMCU, muestra mientras atiende petición
2
3  void setup() {
4      pinMode(LED, OUTPUT); // LED como salida
5      Serial.begin(115200);
6  }
7
8  void loop() {
9      String s, trama; // trama tendrá la trama recibida
10     int inic, fin, opc, i, N, sol;
11     int *v_int; // puntero que apunta al vector de enteros
12     float *v_float, f; // puntero que apunta al vector de enteros
13
14     trama = leerTrama('\n'); // lee la trama
15     digitalWrite(LED, HIGH); // Se enciende mientras procesa
16     s = cortaTrozo(trama, 0, &fin, ':', true); // s (String) recibe la opcion
17     inic=fin; // Apuntamos al principio del siguiente campo de la trama
18     opc = s.toInt(); // La opción (s) se convierte a entero
19     // OBTENER N
20     s = cortaTrozo(trama, inic, &fin, ':', true); // s recibe N como String
21     inic=fin; // Apuntamos al principio del siguiente campo de la trama
22     N = s.toInt(); // N (s) se convierte a entero
23     // Estudio de cada petición
24     if (opc==1) {
25         // v_int contiene el vector de N enteros
26         v_int = obtenListaEnteros(trama, inic, &fin, ':', true, N);
27         sol = sumaListaEnteros(v_int, N); // devuelve la suma N enteros
28         Serial.println( String(sol) ); // Envía a Python el resultado
29         free(v_int); // IMPORTANTE: Libera la memoria de v_int
30     } else if (opc==2) {
31         // v_float contiene el vector de N float
32         v_float = obtenListaFloat(trama, inic, &fin, ':', true, N);
33         inic = fin; // Apuntamos al principio del siguiente campo
34         // s recibe el float a sumar
35         s = cortaTrozo(trama, inic, &fin, ':', true);
36         inic=fin; // Apuntamos al principio del siguiente campo
37         f = s.toFloat(); // s se convierte a float y se almacena en f
38         incrementaListaEn(v_float, N, f); // aumenta cada valor de v_float
39         s = String(""); // s contendrá la trama a enviar a Python
40         for (i=0; i<N; i++) {

```

```

41         s += String(v_float[i]);
42         if (i!=N-1) s += String(':');
43     }
44     Serial.println( s ); // Se envía la trama a Python
45     free(v_float); // IMPORTANTE: Libera la memoria de v_float
46 }
47 digitalWrite(LED, LOW); // Se apaga al terminar de procesar
48 }
49
50 // Funcion que devuelve como String el contenido entre dos "car" (carácter)
51 // comenzando en "inic" y hasta que se encuentra un "car", fin tendrá el
52 // índice donde comienza el siguiente campo
53 String cortaTrozo(String s, int inic, int *fin, char car, bool corta) {
54     int pos_car;
55     pos_car = s.indexOf(car,inic); // posicion de car empezando en inic
56     *fin = pos_car + 1; // fin se asigna a donde comienza el siguiente campo
57     if (pos_car!=-1) return s.substring(inic,pos_car); // Entre inic y fin
58     else if (corta) return s.substring(inic); // Desde inic al final
59     else return String("-1"); // Error, devuelve el String "-1"
60 }
61
62 // Devuelve un vector de N enteros a partir de la trama comenzando en "inic"
63 int *obtenListaEnteros(String linea, int inic, int *fin, char car, bool c, int N) {
64     int *v_int, i;
65     String s;
66     v_int = (int *) malloc(sizeof(int)*N); // Reserva de memoria
67     for(i=0;i<N;i++) { // Para cada trozo
68         s = cortaTrozo(linea, inic, fin, car, c); // s = v_i como String
69         inic=*fin; // Apuntamos al principio del siguiente campo
70         v_int[i] = s.toInt(); // s se convierte a entero y se almacena
71     }
72     return v_int; // Devolución del vector
73 }
74 // Devuelve un vector de tam float a partir de la trama comenzando en "inic"
75 float *obtenListaFloat(String linea, int inic, int *fin, char car, bool c, int N) {
76     float *v_float;
77     int i;
78     String s;
79     v_float = (float *) malloc(sizeof(float)*tam);
80     for(i=0;i<tam;i++) { // Para cada trozo
81         s = cortaTrozo(linea, inic, fin, car, c); // s = v_i como String
82         inic=*fin; // Apuntamos al principio del siguiente campo
83         v_float[i] = s.toFloat(); // s se convierte a float y se almacena
84     }
85     return v_float; // Devolución del vector
86 }
87 // Devuelve la suma de los valores enteros en el vector v
88 int sumaListaEnteros(int v[], int tam) {
89     int i, r = 0;
90     for(i=0;i<tam;i++) {
91         r += v[i];

```

```
92         }
93         return r;
94     }
95     // Incrementa cada valor del vector v en f
96     float incrementaListaEn(float v[], int tam, float M) {
97         int i;
98         for(i=0;i<tam;i++) {
99             v[i] += M;
100         }
101         return M;
102     }
103     // Función que lee una trama terminada en car
104     String leerTrama(char c) {
105         // Espera hasta que haya algo que leer
106         while(Serial.available() < 1) {
107             delay(1);
108         }
109         return Serial.readStringUntil(c); // Recibe info hasta '\n'
110     }
```